```python
class Persona:
    def __init__(self,idPersona,nombre,email,telefono):
        self.idPersona=idPersona
        self.nombre=nombre
        self.email=email
        self.telefono=telefono

    def login(self):
        print(f"Sesión iniciada para: {self.nombre}")

    def logout(self):
        print(f"Sesión terminada para: {self.nombre}")

    def actualizarDatos(self,email,telefono):
        self.email=email
        self.telefono=telefono
        print(f"Datos de {self.nombre} actualizados correctamente")
```

```python
class Usuario(Persona):
    def __init__(self,idPersona,nombre,email,telefono):
        super().__init__(idPersona,nombre,email,telefono)
        self.historialReservas=[]

    def crearReserva(self,pelicula,asientos):
        reserva = {"Pelicula": pelicula, "Asientos": asientos}
        self.historialReservas.append(reserva)
        print(f"Reserva creada con éxito para {pelicula}. Asientos: {', '.join(asientos)}")

    def consultarPromociones(self):
        print("Si tienes algún código y todavía no expira, puedes canjearlo")

    def cancelarReserva(self,pelicula):
        for x in self.historialReservas:
            if x["Pelicula"]==pelicula:
                self.historialReservas.remove(x)
                print(f"La reserva para {pelicula} ha sido cancelada")
                return
        print(f"No se encontró ninguna reserva para {pelicula}")
```

```python
u1 = Usuario(501, "Juan Nieves", "juan@mail.com", "555-201")
u2 = Usuario(502, "María Curiel", "maria@mail.com", "555-202")
u3 = Usuario(503, "Roberto Gómez", "roberto@mail.com", "555-203")
u4 = Usuario(504, "Laura Bozzo", "laura@mail.com", "555-204")
u5 = Usuario(505, "Diego Luna", "diego@mail.com", "555-205")
u6 = Usuario(506, "Sandra Bullock", "sandra@mail.com", "555-206")
u7 = Usuario(507, "Kevin Parker", "kevin@mail.com", "555-207")
u8 = Usuario(508, "Camila Cabello", "camila@mail.com", "555-208")
u9 = Usuario(509, "Bruno Mars", "bruno@mail.com", "555-209")
u10 = Usuario(510, "Taylor Swift", "taylor@mail.com", "555-210")
```

```python
Rol=Enum("Rol",["TAQUILLERO","ADMIN","LIMPIEZA"])

class Empleado(Persona):
    def __init__(self,idEmpleado,nombre,email,telefono,horario,rol:Rol):
        super().__init__(idEmpleado,nombre,email,telefono)
        self.horario=horario
        self.rol=rol

    def marcarEntrada(self):
        print(f"Entrada registrada")

    def gestionarFunciones(self):
        if self.rol.name=="ADMIN":
            siono=input("Agregar nueva función?, SI o NO")
            if siono=="SI":
                print("Película agregada")
            else:
                print("No se ha agregado alguna película")
        else:
            print("Acceso denegado, solo administradores pueden gestionar funciones")
```

```python
e1 = Empleado("E001", "Carlos Ruiz", "carlos@cine.com", "555-101", "08:00-16:00", Rol.ADMIN)
e2 = Empleado("E002", "Ana López", "ana@cine.com", "555-102", "08:00-16:00", Rol.TAQUILLERO)
e3 = Empleado("E003", "Luis Pérez", "luis@cine.com", "555-103", "16:00-00:00", Rol.LIMPIEZA)
e4 = Empleado("E004", "Marta Sosa", "marta@cine.com", "555-104", "16:00-00:00", Rol.ADMIN)
e5 = Empleado("E005", "Jorge Gil", "jorge@cine.com", "555-105", "10:00-18:00", Rol.TAQUILLERO)
e6 = Empleado("E006", "Sofía Mar", "sofia@cine.com", "555-106", "14:00-22:00", Rol.TAQUILLERO)
e7 = Empleado("E007", "Pedro Pic", "pedro@cine.com", "555-107", "06:00-14:00", Rol.LIMPIEZA)
e8 = Empleado("E008", "Lucía Paz", "lucia@cine.com", "555-108", "16:00-00:00", Rol.TAQUILLERO)
e9 = Empleado("E009", "Raúl Rey", "raul@cine.com", "555-109", "08:00-16:00", Rol.LIMPIEZA)
e10 = Empleado("E010", "Elena Sol", "elena@cine.com", "555-110", "12:00-20:00", Rol.ADMIN)
```

```python
class Espacio:
    def __init__(self,idEspacio,nombre,ubicacion):
        self.idEspacio=idEspacio
        self.nombre=nombre
        self.ubicacion=ubicacion

    def verificarDisponibilidad(self):
        if self.ubicacion!="OCUPADO":
            print(f"El lugar: {self.nombre} está disponible")
        else:
            print("El lugar está ocupado")

    def limpiarEspacio(self):
        print(f"El lugar: {self.nombre} ha sido limpiado")
```

```python
class ZonaComida(Espacio):
    def __init__(self,idEspacio,nombre,ubicacion):
        super().__init__(idEspacio,nombre,ubicacion)
        self.listaProductos=[]
        self.stockActual={}

    def venderProducto(self,producto,cantidad):
        if self.stockActual.get(producto,0)>=cantidad:
            print("Producto vendido")
        else:
            print("Stock insuficiente de ",producto)

    def actualizarInventario(self,producto,cantidad):
        self.stockActual[producto]=self.stockActual.get(producto,0)+cantidad
        print(f"Inventario actualizado: {producto} ahora tiene {self.stockActual[producto]} unidades")
```

```python
z1 = ZonaComida(301, "Dulcería Principal", "Lobby Centro")
z2 = ZonaComida(302, "Cafetería Gourmet", "Lobby Izquierda")
z3 = ZonaComida(303, "Bar de Snacks", "Pasillo Salas VIP")
```

```python
Tipo=Enum("Tipo",["2D","3D","IMAX"])

class Sala(Espacio):
    def __init__(self,idEspacio,nombre,ubicacion,capacidadTotal,esVip,tipo:Tipo):
        super().__init__(idEspacio,nombre,ubicacion)
        self.capacidadTotal=capacidadTotal
        self.esVip=esVip
        self.tipo=tipo

    def ajustarAforo(self,capacidad):
        if capacidad<=self.capacidadTotal:
            self.capacidadTotal=capacidad
            print(f"Aforo {self.nombre} ajustado a {capacidad} personas")
        else:
            print("No puedes exceder la capacidad total")

    def obtenerTipoSala(self):
        if self.esVip:
            categoria="VIP"
        else:
            categoria="Tradicional"
        print(f"Esta es una sala {categoria} con tecnología {self.tipo}.")
        return f"{categoria} - {self.tipo}"
```

```python
s1 = Sala(101, "Sala 1", "Planta Baja", 50, False, Tipo.IMAX)
s2 = Sala(102, "Sala 2", "Planta Baja", 50, False, Tipo.IMAX)
s3 = Sala(201, "Sala IMAX 1", "Planta Alta", 120, True, Tipo.IMAX)
s4 = Sala(202, "Sala 3D Plus", "Planta Alta", 80, False, Tipo.IMAX)
s5 = Sala(103, "Sala VIP Lounge", "Planta Baja", 30, True, Tipo.IMAX)
s6 = Sala(104, "Sala Junior", "Planta Baja", 60, False, Tipo.IMAX)
s7 = Sala(203, "Sala Macro XE", "Planta Alta", 150, False, Tipo.IMAX)
```

```python
class Promocion:
    def __init__(self,codigo,descripcion,porcentajeDescuento,fechaExpiracion):
        self.codigo=codigo
        self.descripcion=descripcion
        self.porcentajeDescuento=porcentajeDescuento
        self.fechaExpiracion=fechaExpiracion

    def esValida(self,usuario):
        print(f"Validando promoción {self.codigo} para el usuario {usuario}")
        return True

    def aplicarDescuento(self,monto):
        descuento=monto*(self.porcentajeDescuento/100)
        montoFinal=monto-descuento
        print(f"Descuento aplicado. Total: ${montoFinal}")
        return montoFinal
```

```python
pr1 = Promocion("ESTU20", "20% Estudiantes", 20.0, "2026-12-31")
pr2 = Promocion("MA2X1", "Martes 2x1", 50.0, "2026-06-30")
pr3 = Promocion("SENIOR30", "30% Adulto Mayor", 30.0, "2026-12-31")
pr4 = Promocion("PREMIER", "15% Preventa", 15.0, "2026-05-20")
pr5 = Promocion("CUMPLE", "50% Cumpleañero", 50.0, "2026-12-31")
pr6 = Promocion("SOCIO10", "10% Socio Cine", 10.0, "2026-12-31")
pr7 = Promocion("FAMILIA", "25% Pack Familiar", 25.0, "2026-08-15")
pr8 = Promocion("NOCHE", "20% Función Nocturna", 20.0, "2026-12-31")
pr9 = Promocion("MATINEE", "40% Matinée", 40.0, "2026-09-30")
pr10 = Promocion("PROMOVip", "15% Lunes VIP", 15.0, "2026-11-30")
```

```python
Estado=Enum("Estado",["PENDIENTE","PAGADA","CANCELADA"])

class Reserva:
    def __init__(self,idReserva,usuario,funcion,asientos,montoTotal,estado:Estado):
        self.idReserva=idReserva
        self.usuario=usuario
        self.funcion=funcion
        self.asientos=asientos
        self.montoTotal=montoTotal
        self.estado=estado

    def confirmarPago(self):
        self.estado=Estado.PAGADA
        print("Su pago ha sido confirmado")

    def generarTicket(self):
        if self.estado==Estado.PAGADA:
            print("TICKET DE CINE")
            print(f"Película: {self.funcion}")
            print(f"Cliente: {self.usuario}")
            print(f"ID: {self.idReserva}")
        else:
            print("Tienes que confirmar antes el pago")

    def aplicarPromocion(self,promo:Promocion):
        descuento=self.montoTotal*(promo.porcentajeDescuento/100)
        self.montoTotal-=descuento
        print("Promoción aplicada")
        print(f"Nuevo precio a pagar: ${self.montoTotal}")
```

```python
res1 = Reserva("R001", u1, f1, ["A1", "A2"], 360.0, Estado.PAGADA)
res2 = Reserva("R002", u2, f2, ["C5"], 90.0, Estado.PENDIENTE)
res3 = Reserva("R003", u3, f3, ["B10", "B11"], 220.0, Estado.PAGADA)
res4 = Reserva("R004", u4, f6, ["VIP1"], 250.0, Estado.CANCELADA)
res5 = Reserva("R005", u5, f5, ["D3", "D4"], 260.0, Estado.PAGADA)
res6 = Reserva("R006", u6, f10, ["F1", "F2"], 360.0, Estado.PAGADA)
res7 = Reserva("R007", u7, f7, ["A5", "A6", "A7"], 240.0, Estado.PENDIENTE)
res8 = Reserva("R008", u8, f8, ["G12"], 180.0, Estado.PAGADA)
res9 = Reserva("R009", u9, f9, ["E8"], 90.0, Estado.PAGADA)
res10 = Reserva("R010", u10, f1, ["A3", "A4"], 360.0, Estado.PAGADA)
```

```python
class Funcion:
    def __init__(self,idFuncion,pelicula,sala,horarioInicio,precioBase):
        self.idFuncion=idFuncion
        self.pelicula=pelicula
        self.sala=sala
        self.horarioInicio=horarioInicio
        self.precioBase=precioBase
        self.asientos_ocupados=[]

    def calcularAsientosLibres(self):
        total=self.sala.capacidadTotal
        libres=total-len(self.asientos_ocupados)
        print(f"Asientos libres: {libres} de {total}")
        return libres

    def obtenerDetallesFuncion(self):
        detalles=(
            f"Película: {self.pelicula.titulo}\n"
            f"Sala: {self.sala.nombre} ({self.sala.tipo})\n"
            f"Inicia: {self.horarioInicio}\n"
            f"Precio: ${self.precioBase}"
            )
        print(detalles)
        return detalles
```

```python
f1 = Funcion(1001, p1, s3, "2026-03-05 20:00", 180.0)
f2 = Funcion(1002, p2, s1, "2026-03-05 15:00", 90.0)
f3 = Funcion(1003, p3, s7, "2026-03-05 18:30", 110.0)
f4 = Funcion(1004, p4, s2, "2026-03-05 21:00", 90.0)
f5 = Funcion(1005, p5, s4, "2026-03-06 16:00", 130.0)
f6 = Funcion(1006, p6, s5, "2026-03-06 19:00", 250.0)
f7 = Funcion(1007, p7, s6, "2026-03-06 11:00", 80.0)
f8 = Funcion(1008, p8, s3, "2026-03-06 22:30", 180.0)
f9 = Funcion(1009, p9, s1, "2026-03-07 14:00", 90.0)
f10 = Funcion(1010, p10, s3, "2026-03-07 20:00", 180.0)
```

```python
class Pelicula:
    def __init__(self,titulo,duracion,clasificacion,genero):
        self.titulo=titulo
        self.duracion=duracion
        self.clasificacion=clasificacion
        self.genero=genero

    def obtenerSinopsis(self):
        return f"Sinopsis detallada de {self.titulo} ({self.genero})"

    def esAptaParaTodoPublico(self):
        if self.clasificacion=="A":
            print("Es apta para todo público")
        else:
            print("No es apta para todo público")
```

```python
p1 = Pelicula("Dune: Part Two", 166, "B", "Sci-Fi")
p2 = Pelicula("Kung Fu Panda 4", 94, "A", "Animación")
p3 = Pelicula("Oppenheimer", 180, "C", "Drama")
p4 = Pelicula("The Batman", 176, "B", "Acción")
p5 = Pelicula("Spiderman: Across the Spiderverse", 140, "A", "Animación")
p6 = Pelicula("Pobres Criaturas", 141, "C", "Comedia Negra")
p7 = Pelicula("Super Mario Bros", 92, "A", "Aventura")
p8 = Pelicula("John Wick 4", 169, "C", "Acción")
p9 = Pelicula("Barbie", 114, "B", "Comedia")
p10 = Pelicula("Avatar: The Way of Water", 192, "B", "Fantasía")
```

```
PRUEBA: GESTI►N DE FUNCIONES
Pel►cula: Dune: Part Two
Sala: Sala IMAX 1 (Tipo.IMAX)
Inicia: 2026-03-05 20:00
Precio: $180.0

PRUEBA: CREACI►N DE RESERVA
Sesi►n iniciada para: Juan Nieves
Reserva creada con ►xito para Dune: Part Two. Asientos: A1, A2

PRUEBA: PROMOCI►N Y PAGO
Validando promoci►n ESTU20 para el usuario <models.Usuario object at 0x0000020154368D70>
Promoci►n aplicada
Nuevo precio a pagar: $288.0
Su pago ha sido confirmado
TICKET DE CINE
Pel►cula: <models.Funcion object at 0x0000020154368B90>
Cliente: <models.Usuario object at 0x0000020154368D70>
ID: R001

PRUEBA: DISPONIBILIDAD Y LIMPIEZA
El lugar: Sala IMAX 1 est► disponible
El lugar: Sala IMAX 1 ha sido limpiado

PRUEBA: ZONA DE COMIDA
Inventario actualizado: Palomitas Grandes ahora tiene 50 unidades
Producto vendido
```